Державний університет «Житомирська політехніка»
Факультет інформаційно-комп'ютерних технологій (ФІКТ)
Кафедра комп'ютерної інженерії та штучного інтелекту (123)
Освітній ступінь: Баклавр (І курс)

ЗВІТ

з лабораторної роботи № 4

Дисципліна: Програмування

Тема: «Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git»

Варіант № 2

Виконала: Веремійчук В.Є.

студентка І курсу академічної групи Кі-26-1

Перевірив:

Лисенко М. С.

Житомир — 2026

1. Мета роботи та індивідуальне завдання

Мета:

Опанувати принципи функціональної декомпозиції програм: проектування чистих підпрограм із чітким розділенням на прототипи (оголошення) та реалізацію (визначення).

Дослідити механізм роботи автоматичних локальних змінних на стеку та використання специфікатора static для збереження внутрішнього стану функцій між послідовними викликами.

Опанувати низькорівневе маніпулювання даними на рівні окремих розрядів за допомогою порозрядних операцій & , | , ^ , ~ , << , >> .

Засвоїти чотири канонічні інженерні операції над апаратними регістрами мікропроцесора: встановлення, скидання, інвертування та тестування стану бітів за допомогою динамічно сформованих масок.

Опанувати командний процес інтеграції коду через створення, рецензування та злиття Pull Request (PR) на платформі GitHub.

Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

Завдання 1. Обчислення факторіала.
Розробити програму на C, у якій обчислення факторіала виконується окремою функцією calculate_factorial(int n). Функція має повертати n! для невід’ємного цілого n, використовувати статичну локальну змінну total_multiplications для накопичення загальної кількості множень між викликами та виводити її поточне значення. Для від’ємного аргументу передбачено повернення 0. У main() послідовно перевіряються значення 5, 3, 7, 0, 4.

Завдання 2. Побітова обробка регістра
Початкове значення 32-бітового регістра gpio_reg дорівнює 0x00008080. Реалізувати окремі функції встановлення біта set_bit, очищення біта clear_bit, інвертування біта toggle_bit, перевірки біта check_bit та виведення регістра у двійковому форматі. Послідовно встановити біт 3, очистити біт 7, інвертувати біт 12 і перевірити стан біта 15. Двійкове представлення виводити групами по чотири біти, додатково показуючи шістнадцяткове значення.

Завдання 3. Пакування телеметричних даних
Описати структуру TelemetryData та реалізувати функції pack_telemetry і unpack_telemetry. У 16-бітовому слові розмістити поля: sensor_code — біти 0–7

(8 бітів), work_mode — біти 8–11 (4 біти), alarm_flag — біт 12 (1 біт), channel_num — біти 13–15 (3 біти). Виконати пакування заданих значень sensor=0xA5, mode=0x0C, alarm=0x01, channel=0x06, а потім розпакувати слово й перевірити відновлення полів.

2. Математичний аналіз та пояснення бітових масок

2.1. Завдання 1 — факторіал і лічильник множень

Факторіал невід'ємного цілого числа визначається як:

$n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n, \quad 0! = 1.$

Для циклу використовується рекурентне накопичення: $F_0 = 1$; $F_i = F_{i-1} \cdot i$ для i = 1, 2, …, n. На кожній ітерації виконується одне множення, тому кількість множень одного виклику для $n \geq 0$ дорівнює n. Статична змінна total_multiplications зберігає суму між викликами. Для послідовності 5, 3, 7, 0, 4 накопичені значення становлять 5, 8, 15, 15, 19.

Для n = 0 цикл не виконується, fact залишається рівним 1, а кількість множень не змінюється. Для n < 0 функція повертає 0 до початку циклу. У програмі не передбачено окремої перевірки переповнення unsigned long long, тому коректність числового результату обмежується діапазоном цього типу.

2.2. Завдання 2 — маски та операції над бітами

Для номера біта b створюється маска M = 1U << b. Операція встановлення використовує OR: reg = reg | M; очищення — AND із запереченою маскою: reg = reg & ~M; інвертування — XOR: reg = reg ^ M. Перевірка повертає 1, якщо reg & M не дорівнює нулю, інакше 0.

Початковий регістр 0x00008080 має встановлені біти 15 і 7. Послідовність змін:

Крок	**Операція**	**Результат (hex)**	**Пояснення**
Початковий стан	—	0x00008080	Біти 15 та 7 встановлені
Встановити біт 3	OR з 0x00000008	0x00008088	Біт 3 стає 1
Очистити біт 7	AND з ~0x00000080	0x00008008	Біт 7 стає 0
Інвертувати біт 12	XOR з 0x00001000	0x00009008	Біт 12 змінюється з 0 на 1
Перевірити біт 15	AND з 0x00008000	1	Біт 15 залишається встановленим

Під час виведення цикл проходить від біта 31 до біта 0. Вираз (reg >> i) & 1 виділяє поточний біт. Пробіл після кожної групи з чотирьох бітів полегшує читання двійкового числа.

2.3. Завдання 3 — схема пакування

Кожне поле маскується до дозволеної ширини, після чого зсувається на позицію свого поля та об’єднується операцією OR:

packet = (sensor & 0xFF) | ((mode & 0x0F) << 8) | ((alarm & 0x01) << 12) | ((channel & 0x07) << 13).

Для заданих даних: sensor=0xA5; mode=0x0C; alarm=1; channel=6. Після зсувів отримуємо 0x00A5, 0x0C00, 0x1000 та 0xC000. Їх побітове OR дає 0xDCA5. Під час розпакування поля виділяються зсувом праворуч і масками 0xFF, 0x0F, 0x01 та 0x07 відповідно. Це повертає початкові значення 165, 12, 1 і 6.

3. Опис алгоритмів та логічних рішень

3.1. Завдання 1

У main() цикл for перебирає п’ять наперед заданих тестових чисел. Для кожного числа викликається calculate_factorial, після чого результат факторіала друкується в консоль. Усередині функції умовний оператор if відсікає від’ємне значення. Для невід’ємного n цикл for послідовно множить поточний добуток на і та збільшує статичний лічильник. Цикл має скінченну кількість ітерацій n; за n=0 він не запускається.

3.2. Завдання 2

main() ініціалізує регістр і виконує операції над конкретними номерами бітів у заданому порядку. Функції set_bit, clear_bit і toggle_bit змінюють регістр через вказівник, тому зміни зберігаються у змінній gpio_reg. check_bit повертає логічний результат у типі uint8_t. У print_binary цикл із лічильником і від 31 до 0 завершується після обробки нульового біта. Окремого введення користувачем немає.

3.3. Завдання 3

main() задає чотири початкові поля, друкує їх, викликає pack_telemetry, показує отримане 16-бітове слово, а потім викликає unpack_telemetry та друкує відновлені поля. У функціях пакування/розпакування застосовано маски та зсуви. Умовних операторів і циклів у цих двох функціях немає.

3.4. Захисні заходи та межі перевірки

Надані програми використовують фіксовані тестові значення й не читають дані з клавіатури. У task1.c передбачено перевірку n < 0, однак у main() від'ємне значення не тестується. У task2.c номери бітів задані константами в межах 0–31; функції не перевіряють допустимість bit, тому для загального використання потрібна додаткова перевірка bit < 32. У task3.c маски обмежують значення полів їхньою шириною, але окремого повідомлення про некоректні значення немає — надлишкові біти відкидаються.

Зациклення не виникає: усі цикли мають лічильники з чітко визначеними межами. У task1.c цикл завершується після i > n, а у task2.c — після переходу від i=0 до i=-1. Вимоги про скріншоти від'ємного кроку або виходу за межі області збіжності не відповідають цим трьом програмам: у них немає кроку числового табулювання чи області збіжності. Такі сценарії не можна коректно продемонструвати без зміни індивідуального завдання. Для звіту доцільно додати скріншоти фактичного запуску кожної програми; окремо можна показати перевірку n<0 лише після додавання такого тесту до main().

4. Лістинг вихідного коду

4.1. Файл task1.c

```c
#include <stdio.h>

unsigned long long calculate_factorial(int n);

int main(void) {
    int test_numbers[] = {5, 3, 7, 0, 4};
    int count = sizeof(test_numbers) / sizeof(test_numbers[0]);

    for (int i = 0; i < count; i++) {
        int num = test_numbers[i];
        unsigned long long result = calculate_factorial(num);
        printf("%d! = %llu\n", num, result);
    }

    return 0;
}

unsigned long long calculate_factorial(int n) {
    static unsigned long long total_multiplications = 0;

    if (n < 0) {
        return 0;
    }

    unsigned long long fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
        total_multiplications++;
    }

    printf("Total multiplications count: %llu\n", total_multiplications);

    return fact;
}
```

4.2. Файл task2.c

```c
#include <stdio.h>
#include <stdint.h>

void set_bit(uint32_t *reg, uint8_t bit);
void clear_bit(uint32_t *reg, uint8_t bit);
void toggle_bit(uint32_t *reg, uint8_t bit);
uint8_t check_bit(uint32_t reg, uint8_t bit);
void print_binary(uint32_t reg);

int main(void) {
    uint32_t gpio_reg = 0x00008080;

    printf("Initial register state:\n");
    print_binary(gpio_reg);
    printf("\n");

    set_bit(&gpio_reg, 3);
    printf("After setting bit 3:\n");
    print_binary(gpio_reg);
    printf("\n");

    clear_bit(&gpio_reg, 7);
    printf("After clearing bit 7:\n");
    print_binary(gpio_reg);
    printf("\n");

    toggle_bit(&gpio_reg, 12);
    printf("After toggling bit 12:\n");
    print_binary(gpio_reg);
    printf("\n");

    uint8_t state = check_bit(gpio_reg, 15);
    printf("Bit 15 state: %u\n", state);

    return 0;
}
```

```c
void set_bit(uint32_t *reg, uint8_t bit) {
    *reg |= (1U << bit);
}

void clear_bit(uint32_t *reg, uint8_t bit) {
    *reg &= ~(1U << bit);
}

void toggle_bit(uint32_t *reg, uint8_t bit) {
    *reg ^= (1U << bit);
}

uint8_t check_bit(uint32_t reg, uint8_t bit) {
    return (reg & (1U << bit)) ? 1 : 0;
}

void print_binary(uint32_t reg) {
    printf("BIN: ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (reg >> i) & 1);
        if (i % 4 == 0 && i != 0) {
            printf(" ");
        }
    }
    printf(" (0x%08X)\n", reg);
}

```

4.3. Файл task3.c

```c
#include <stdio.h>
#include <stdint.h>

typedef struct {
    uint8_t sensor_code;
    uint8_t work_mode;
    uint8_t alarm_flag;
    uint8_t channel_num;
} TelemetryData;

uint16_t pack_telemetry(uint8_t sensor, uint8_t mode, uint8_t alarm, uint8_t channel);
TelemetryData unpack_telemetry(uint16_t packet);

int main(void) {
    uint8_t sensor = 0xA5;
    uint8_t mode = 0x0C;
    uint8_t alarm = 0x01;
    uint8_t channel = 0x06;

    printf("Original values:\n");
    printf("Sensor: %u, Mode: %u, Alarm: %u, Channel: %u\n\n", sensor, mode, alarm, channel);

    uint16_t packet = pack_telemetry(sensor, mode, alarm, channel);
    printf("Packed word: 0x%04X\n\n", packet);

    TelemetryData restored = unpack_telemetry(packet);
    printf("Unpacked values:\n");
    printf("Sensor: %u, Mode: %u, Alarm: %u, Channel: %u\n",
           restored.sensor_code, restored.work_mode, restored.alarm_flag, restored.channel_num);

    return 0;
}
```

```c
uint16_t pack_telemetry(uint8_t sensor, uint8_t mode, uint8_t alarm, uint8_t channel) {
    uint16_t packet = 0;
    packet |= (sensor & 0xFF);
    packet |= ((mode & 0x0F) << 8);
    packet |= ((alarm & 0x01) << 12);
    packet |= ((channel & 0x07) << 13);
    return packet;
}

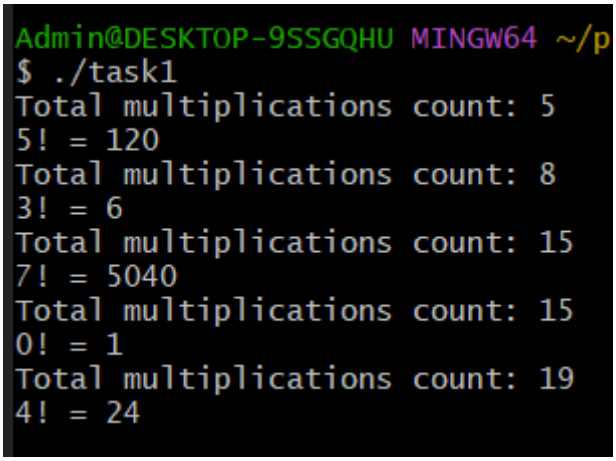
TelemetryData unpack_telemetry(uint16_t packet) {
    TelemetryData data;
    data.sensor_code = packet & 0xFF;
    data.work_mode = (packet >> 8) & 0x0F;
    data.alarm_flag = (packet >> 12) & 0x01;
    data.channel_num = (packet >> 13) & 0x07;
    return data;
}
```

5. Результати тестування та верифікація

5.1. Завдання 1

Під час послідовного запуску функції лічильник множень накопичується між викликами. Нижче наведено очікувані результати для заданого набору.

n	n!	Множень у виклику	Лічильник після виклику
5	120	5	5
3	6	3	8
7	5040	7	15
0	1	0	15
4	24	4	19



5.2. Завдання 2

Етап	Значення регістра	Перевірка
Початковий	0x00008080	Біти 15 та 7 дорівнюють 1
Після set_bit(3)	0x00008088	Біт 3 встановлено
Після clear_bit(7)	0x00008008	Біт 7 очищено
Після toggle_bit(12)	0x00009008	Біт 12 інвертовано
check_bit(15)	1	Біт 15 встановлений

```
Admin@DESKTOP-9SSGQHU MINGW64 ~/projects/programming-sem1/la
$ ./task2
Initial register state:
BIN: 0000 0000 0000 0000 1000 0000 1000 0000 (0x00008080)

After setting bit 3:
BIN: 0000 0000 0000 0000 1000 0000 1000 1000 (0x00008088)

After clearing bit 7:
BIN: 0000 0000 0000 0000 1000 0000 0000 1000 (0x00008008)

After toggling bit 12:
BIN: 0000 0000 0000 0000 1001 0000 0000 1000 (0x00009008)

Bit 15 state: 1
```

5.3. Завдання 3

Поле	Початкове значення	Значення після розпакування
Sensor	165 (0xA5)	165
Mode	12 (0x0C)	12
Alarm	1	1
Channel	6	6
Packed word	—	0xDCA5

```
Admin@DESKTOP-9SSGQHU MINGW64 ~/projects/programming
$ ./task3
Original values:
Sensor: 165, Mode: 12, Alarm: 1, Channel: 6

Packed word: 0xDCA5

Unpacked values:
Sensor: 165, Mode: 12, Alarm: 1, Channel: 6
```

Результат пакування 0xDCA5 відповідає бітовому розподілу 110 | 1 | 1100 | 10100101 (біти 15–13 | 12 | 11–8 | 7–0). Після розпакування всі чотири поля збігаються з початковими значеннями, отже перевірку відновлення пройдено.

6. Висновки

У ході лабораторної роботи реалізовано та перевірено три окремі програми мовою C. У першій програмі факторіал обчислюється циклічно, а статична локальна змінна зберігає сумарну кількість множень між викликами; результати для тестових значень відповідають математичному визначенню факторіала. У другій програмі продемонстровано встановлення, очищення, інвертування та перевірку бітів 32-бітового регістра за допомогою масок і побітових операторів. У третій програмі виконано пакування чотирьох телеметричних полів у 16-бітове слово та їх подальше відновлення; отримане слово 0xDCA5 і відновлені значення відповідають заданій структурі полів. Отже, мету роботи досягнуто: закріплено навички функціонального розбиття програми, циклів, статичних змінних і побітової обробки даних. Зауваженням є те, що захист меж бітового індексу та повідомлення про значення полів поза діапазоном у наданому коді не реалізовані, а сценарії з від’ємним кроком чи областю збіжності до цих завдань не належать.